

ISIT312/ISIT912 Big Data Management

Spring 2025

Spark Stream Processing (DStream) and

Spark Structured Streaming

In this practice, you will learn Spark Stream Processing (DStream) and Spark Structured Streaming.

Warning: To copy the fragments of code from this document to your working Terminal use *evince pdf viewer*.

Software Installation and Setup

If you use a system installed in 3.230 i.e. a laboratory room for ISIT312/912, then you do not need to install VirtualBox.

If you use your system (laptop) then start from installation of the latest version of VirtualBox and the extension pack (optional). The distribution packages are available at <https://www.virtualbox.org/wiki/Downloads> for Windows, MacOS (Intel processor only) and Linux.

If you use your system (laptop) then you must copy a file `BigData-Ubuntu22.04.4-Hadoop3.3.6-P3-Spark3.5.1.ova` located at `C:\Program Files (x86)\ISIT312` and `ISIT912` (if you cannot find the file in this location please ask a tutor about correct location). You can also download the file from Moodle.

Next, start VirtualBox and import the file. An instruction is in

https://docs.oracle.com/cd/E26217_01/E26796/html/qs-import-vm.html

or you can just double-click at the `ova` file.

After the import is completed, a virtual machine named `BigData-Ubuntu22.04.4-Hadoop3.3.6-P3-Spark3.5.1` appears in the VirtualBox.

If you use a system installed in 3.320 then start VirtualBox and import a file `BigData-Ubuntu22.04.4-Hadoop3.3.6-P3-Spark3.5.1.ova` located at `C:\Program Files (x86)\ISIT312` and `ISIT912` folder into VirtualBox. An instruction is in

https://docs.oracle.com/cd/E26217_01/E26796/html/qs-import-vm.html

or you can just double-click at the `ova` file.

After the import is completed, a virtual machine named `BigData-Ubuntu22.04.4-Hadoop3.3.6-P3-Spark3.5.1` appears in the VirtualBox.

Laboratory Instructions

Start a virtual machine `BigData-Ubuntu22.04.4-Hadoop3.3.6-P3-Spark3.5.1`

Note, that both the account name and the password are: `bigdata`

(0) Start Shell

After you log on the Ubuntu system with a user name `bigdata` and password `bigdata`, start a Terminal window with `Ctrl + Alt + T` or use the fifth from top icon located at a vertical stripe (sidebar) on the left-hand side of the screen.

The documents `LinuxCommandLineCheatSheet.pdf` and `Efficient-Linux-at-the-Command-Line-ch4.pdf` available through [Readings](#) link in [Resources](#) section in `linux` folder on Moodle contain more information on how to use Linux Shell in Terminal window.

You can use the Terminal window to interact with Hadoop. A simple hint that may make your life much easier is to use "Up" and "Down" keys on a keyboard to navigate through the commands already processed in Terminal window.

Copy and Paste in Terminal window can be done with the keys `Ctrl/Insert` and with the keys `Shift/Insert` or with a mouse rightclick.

Now you can interact with Hadoop.

(1) Hadoop Initialisation

Now you can start Hadoop. Process a shell script `start-all.sh` in Terminal window in the following way:

```
$HADOOP_HOME/sbin/start-all.sh
```

The system will display few warnings and the following messages:

```
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [bigdata-VirtualBox]
Starting resourcemanager
Starting nodemanagers
```

After you ignore the warning messages you can view the running processes with a command:

```
jps
```

The following results should be returned (note that the process numbers may be different):

```
819 NodeManager
2501 SecondaryNameNode
2310 DataNode
4391 Jps
2187 NameNode
2700 ResourceManager
```

(2) Spark Initialisation

Now you can start Spark. Process a shell script `start-all.sh` in Terminal window in the following way:

```
$SPARK_HOME/sbin/start-all.sh
```

The system will display few warnings and the following messages:

```
starting org.apache.spark.deploy.master.Master, logging to
/home/bigdata/spark-3.5.1-bin-hadoop3/logs/spark-bigdata-
org.apache.spark.deploy.master.Master-1-bigdata-VirtualBox.out
localhost: starting org.apache.spark.deploy.worker.Worker, logging
to /home/bigdata/spark-3.5.1-bin-hadoop3/logs/spark-bigdata-
org.apache.spark.deploy.worker.Worker-1-bigdata-VirtualBox.out
```

After you ignore the warning messages you can view the running processes with a command:

jps

The following results should be returned (note that the process numbers may be different):

```
3634 SecondaryNameNode
3955 NodeManager
3447 DataNode
7207 Jps
4329 Master
4441 Worker
3310 NameNode
3839 ResourceManager
```

(3) How to start and interact with Spark in the Shark Shell?

To start the PySpark shell, process the following command in a Terminal window.

pyspark

PySpark is the Python Application Programming Interface for Apache Spark. PySpark displays the following messages.

```
Python 3.10.12 (main, Nov 20 2023, 15:14:05) [GCC 11.4.0] on linux
Type "help", "copyright", "credits" or "license" for more
information.
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR,
use setLogLevel(newLevel).
Welcome to
```

version 3.5.1

```
Using Python version 3.10.12 (main, Nov 20 2023 15:14:05)
Spark context Web UI available at http://10.0.2.15:4040
Spark context available as 'sc' (master = local[*], app id = local-1716025752494).
SparkSession available as 'spark'.
```

(4) How to process data streams with Spark Stream Processing (DStream)

Open new Terminal window and start netcat utility in the following way.

nc -lk 9999

Open yet another Terminal window and use a text editor to create a file `dstream.py` with the following contents.

```
from pyspark import SparkContext
from pyspark.streaming import StreamingContext

sc = SparkContext("local[2]", "NetworkWordCount")
ssc = StreamingContext(sc, 1)

lines = ssc.socketTextStream("localhost", 9999)
words = lines.flatMap(lambda line: line.split(" "))
pairs = words.map(lambda word: (word, 1))
wordCounts = pairs.reduceByKey(lambda x, y: x + y)
wordCounts.pprint()

ssc.start()
ssc.awaitTermination()
```

When ready, save a file `dstream.py`.

Move to Terminal window where `netcat` utility is running and type the following lines.

```
aa aa aa aa bb bb
bb bb bb aa aa
```

Move to Terminal window earlier used for creation of a file `dstream.py`, i.e. Terminal window where the current folder that contains a file `dstream.py`.

Start a data stream processing application `dstream.py` in the following way.

```
$SPARK_HOME/bin/spark-submit dstream.py localhost 9999
```

Check the results displayed by `dstream.py`.

Move to Terminal window where `netcat` utility is running and enter more lines.

Again check the results displayed by `dstream.py`.

Use `CTRL C` to stop `dstream.py` and then use `CTRL C` to stop `netcat` utility.

(5) How to process data streams with Spark Structured Stream Processing

First we use PySpark to demonstrate structured stream processing. Like before, open new Terminal window and start `netcat` utility.

```
nc -lk 9999
```

Move to Terminal window where PySpark shell was started in step (3).

Insert in front of PySpark prompt `>>>` the code listed below and wait a moment until PySpark processes the code.

```
from pyspark.sql.functions import explode
from pyspark.sql.functions import split

lines = spark.readStream.format("socket")
```

```

        .option("host", "localhost")
        .option("port", 9999).load()

words = lines.select( explode( split(lines.value, " ") )
                    .alias("word"))

wordCounts = words.groupBy("word").count()

query = wordCounts.writeStream.outputMode("complete")
        .format("console").start()

```

After a pretty long while the structured streaming application displays `Batch 0` with an empty table.

Move to `netcat` window and type few lines with words..

Return to a window where structured streaming application is running to see `Batch: 1`.

Move to `netcat` window and again type few words.

Return to a window where structured streaming application is running to see `Batch: 2`. It contains aggregated results from `Batch 1` window with the words recently typed in `netcat` window.

Type `CTRL C` in both `netcat` and `PySpark` windows to stop streaming.

A standalone version of the same structured streaming application is given below.

```

from pyspark.sql import SparkSession
from pyspark.sql.functions import explode
from pyspark.sql.functions import split

spark = SparkSession.builder.appName("StructuredNetworkWordCount")
                        .getOrCreate()
lines = spark.readStream.format("socket")
                        .option("host", "localhost")
                        .option("port", 9999).load()
words = lines.select( explode( split(lines.value, " ") )
                    .alias("word"))
wordCounts = words.groupBy("word").count()
query = wordCounts.writeStream.outputMode("complete")
        .format("console")
        .start()

query.awaitTermination()

```

Use a text editor to create a file `sstream.py` with a standalone structured streaming application listed above.

Start `netcat` in the same Terminal window as before.

Process the application in another Terminal widow where a file `sstream.py` is located in the current folder.

```
$SPARK_HOME/bin/spark-submit sstream.py localhost 9999
```

Wait for a while until the application displays `Batch: 0`. Then type few lines in `netcat` window. Wait for a while until the application displays `Batch: 1`, and so on.

Use `CTRL C` to stop the application and to stop `netcat`.

(6) Using DataFrames for Spark Structured Stream Processing

In the next example we filter out from a stream the numbers less than 100 and we count the repetitions of the numbers. This time we use DataFrames to collect, to filter the number and to count the numbers.

We use `PySpark` shell to implement the application.

First, start `netcat` in the same window as before.

```
nc -lk 9999
```

Next, we enter the following lines in front of `>>>` prompt in `PySpark` window.

```
socketDF = spark.readStream
            .format("socket")
            .option("host", "localhost")
            .option("port", 9999).load()
numbers = socketDF.select(socketDF.value.cast("Int")
                        .alias("number"))
numbersge100= numbers.filter(numbers.number>=100)
numberCounts = numbersge100.groupBy("number").count()
query = numberCounts.writeStream.outputMode("complete")
                        .format("console").start()
```

Wait until the application displays `Batch 0:`.

Next type a number greater or equal to 100 in `netcat` window and press Enter key.

Wait until the application displays `Batch 1:`.

Next type a number less than 100 in `netcat` window and press Enter key.

Wait until the application displays `Batch 2: ...` and so on.

(7) Processing a stream of files with SQL

In the next exercise we show how to use SQL for data stream processing. Like in the previous exercise we process a stream of `csv` files being uploaded to home folder on HDFS.

Use a text editor to create a `csv` file `grades1.csv`.

```
James,20
Alice,55
Harry,0
Charlie,99
```

Copy `csv` file `grades1.csv` into the files `grades2.csv` and `grades 3 .csv`.

Next use a text editor to slightly change the values of grades in the files `grades2.csv` and `grades3.csv`.

Start PySpark in Terminal window and enter the following lines in front of `>>>` prompt.

```
from pyspark.sql.functions import StructType
userSchema = StructType().add("name", "string")
                        .add("score", "integer")
gradesDF = spark.readStream.schema(userSchema)
                .csv("/user/bigdata")
gradesDF.createOrReplaceTempView("grades")
sqlDF=spark.sql(
    "select count(*), max(score), min(score), avg(score) from grades"
)
query = sqlDF.writeStream
                .outputMode("complete").format("console").start()
```

Next, upload a file `grades1.csv` to your home folder on HDFS.

```
hdfs dfs -put grades.csv
```

Return to PySpark window and wait for Batch 0: . Each batch created by the application finds the total number of scores in `csv` files uploaded to a home folder on HDFS, the highest score, the lowest score and an average of all scores (see SQL statement used as an argument of `sql` method applied to `spark` object).

Next, upload a file `grades2.csv` to a home folder on HDFS and check Batch 1:.

Finally, upload a file `grades3.csv` to a home folder on HDFS and check Batch 2:.

(8) Processing a stream of data over window-based aggregations (output mode complete)

In the next experiment, we show how to process data streams over window-based aggregations. Up to now the aggregate values, like for example counts, were maintained for an entire period of time from the beginning of data streaming up to ... theoretical infinity. In a case of window-based aggregations, aggregate values are maintained for each window the event-time of a data item falls into.

Open Terminal window and start `netcat`.

```
nc -lk 9999
```

Next, we enter the following lines in front of `>>>` prompt in PySpark window.

```
from pyspark.sql.functions import window
stringDF = spark.readStream
                .format("socket")
                .option("host", "localhost")
                .option("port", 9999)
                .option("includeTimestamp", "true").load()
windowedCounts = stringDF.groupBy(window(stringDF.timestamp,
    "1 minute", "1 minute"), stringDF.value).count()
```

```

                                .orderBy("window")
query = windowedCounts.writeStream
                        .outputMode("complete")
                        .format("console").start()

```

Note an option `.option("includeTimestamp", "true")` applied to spark object.

The parameters of a sliding window are set by `window(stringDF.timestamp, "1 minute", "1 minute")`. The parameters mean that aggregations will be performed within 1 minute time window (the first parameter) and a time window will slide every single minute (the second parameter).

To start a process type

```
aaa
```

in netcat window and press Enter key.

The first Batch 0: is empty.

Batch 1: contains

```

Batch: 1
-----
+-----+-----+-----+
|           window|value|count|
+-----+-----+-----+
|{2024-06-07 17:38...|  aaa|    1|
+-----+-----+-----+

```

Now, wait 1 minute and then type

```
bbb
```

in netcat window and press Enter key.

Batch 2: contains

```

-----
Batch: 2
-----
+-----+-----+-----+
|           window|value|count|
+-----+-----+-----+
|{2024-06-07 17:38...|  aaa|    1|
|{2024-06-07 17:40...|  bbb|    1|
+-----+-----+-----+

```

Now, wait 1 minute and then quickly type

```

aaa
aaa
bbb

```


in netcat window and press `Enter` key to fit 3 data items in the same time window.

Batch 3: contains

```
-----
Batch: 3
-----
+-----+-----+-----+
|           window|value|count|
+-----+-----+-----+
|{2024-06-07 17:48...|  aaa|    1|
|{2024-06-07 17:49...|  bbb|    1|
|{2024-06-07 17:50...|  aaa|    1|
+-----+-----+-----+
```

Note that the latest `aaa` is not aggregated because it was provided in different time window. The same applies to the second `aaa` and `bbb` in Batch 4:.

```
-----
Batch: 4
-----
+-----+-----+-----+
|           window|value|count|
+-----+-----+-----+
|{2024-06-07 17:48...|  aaa|    1|
|{2024-06-07 17:49...|  bbb|    1|
|{2024-06-07 17:50...|  bbb|    1|
|{2024-06-07 17:50...|  aaa|    2|
+-----+-----+-----+
```

Only data item recorded in the same time window are aggregated

To stop processing a data stream enter `CTRL C` in both netcat and PySpark windows.

In the next experiment we consider 2 minutes time window moving every single minute.

Open Terminal window and start netcat.

```
nc -lk 9999
```

Next, we enter the following lines in front of `>>>` prompt in PySpark window.

```
from pyspark.sql.functions import window
stringDF = spark.readStream
    .format("socket")
    .option("host", "localhost")
    .option("port", 9999)
    .option("includeTimestamp", "true").load()
windowedCounts = stringDF.groupBy(window(stringDF.timestamp,
    "2 minutes", "1 minute"), stringDF.value).count()
    .orderBy("window")
query = windowedCounts.writeStream
    .outputMode("complete")
    .format("console").start()
```

The parameters of a sliding window are set by `window(stringDF.timestamp, "2 minutes", "1 minute")`. The parameters mean that aggregations will be performed in 2 minutes time window (the first parameter) and a time window will slide every single minute (the second parameter).

To start a process type

```
aaa
```

in netcat window and press Enter key.

The first Batch 0: is empty.

Batch 1: contains

```
-----
Batch: 1
-----
+-----+-----+-----+
|               window|value|count|
+-----+-----+-----+
|{2024-06-07 22:27...|  aaa|    1|
|{2024-06-07 22:28...|  aaa|    1|
+-----+-----+-----+
```

The results consist of two rows. An aggregation performed at the original position of a time window at 22:27 and an aggregation performed after a window slide one minute forward at 22:28.

Next, type

```
bbb
```

in netcat window and press Enter key.

Batch 2: contains

```
-----
Batch: 2
-----
+-----+-----+-----+
|               window|value|count|
+-----+-----+-----+
|{2024-06-07 22:27...|  aaa|    1|
|{2024-06-07 22:28...|  aaa|    1|
|{2024-06-07 22:28...|  bbb|    1|
|{2024-06-07 22:29...|  bbb|    1|
+-----+-----+-----+
```

The results consist of two new rows. An aggregation performed at a position of a time window at 22:28 and an aggregation performed after a window slide one minute forward at 22:29.

Next, type

aaa
aaa
bbb

in netcat window and press Enter key.

Batch 3: contains

Batch: 3

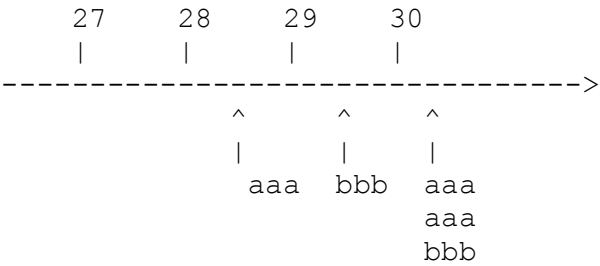
window	value	count
{2024-06-07 22:27...}	aaa	1
{2024-06-07 22:28...}	aaa	1
{2024-06-07 22:28...}	bbb	1
{2024-06-07 22:29...}	aaa	1
{2024-06-07 22:29...}	bbb	1
{2024-06-07 22:30...}	aaa	1

One of data items aaa arrived in a time widow located 22:29 and the other in a time window at 22:30.

Batch: 4

window	value	count
{2024-06-07 22:27...}	aaa	1
{2024-06-07 22:28...}	aaa	1
{2024-06-07 22:28...}	bbb	1
{2024-06-07 22:29...}	aaa	2
{2024-06-07 22:29...}	bbb	2
{2024-06-07 22:30...}	aaa	2
{2024-06-07 22:30...}	bbb	1

After aggregation a time window at 22:29 contains 2 data items aaa and 2 data items bbb. A time window at 22:30 contains 2 data items aaa and one data item bbb. Please see a diagram below.



(9) Processing a stream of data over window-based aggregations with SQL (output mode complete)

In the next experiment, we use SQL to process data items in a time window in an output mode complete. It means that aggregation is on all data items that arrived during a given time window. A time window size is 1 minute and the windows slides every 1 minute.

Open Terminal window and start `netcat`.

```
nc -lk 9999
```

Next, we enter the following lines in front of `>>>` prompt in PySpark window.

```
from pyspark.sql.functions import window
stringDF = spark.readStream.format("socket")
                        .option("host", "localhost")
                        .option("port", 9999)
                        .option("includeTimestamp", "true")
                        .load()
windowedCounts = stringDF.groupBy(
    window(stringDF.timestamp, "1 minute", "1 minute"), stringDF.value)
                        .count().orderBy("window")
windowedCounts.createOrReplaceTempView("sqlCounts")
wCounts = spark.sql("SELECT window,value FROM sqlCounts")
query = wCounts.writeStream
                        .outputMode("complete").format("console").start()
```

To start a process type

```
aaa
```

in `netcat` window and press `Enter` key.

Next, wait 1 minute and then type

```
bbb
```

in `netcat` window and press `Enter` key.

Next, wait 1 minute and then quickly type

```
aaa
aaa
bbb
```

in `netcat` window and press `Enter` key to fit 3 data items in the same time window.

Only data item recorded in the same time window are aggregated.

The results are the same as in the first experiment in section (7).

To stop processing a data stream enter `CTRL C` in both `netcat` and PySpark windows.

(10) Processing a stream of data over window-based aggregations (output mode update)

Open Terminal window and start `netcat`.

```
nc -lk 9999
```

Next, we enter the following lines in front of >>> prompt in PySpark window.

```
from pyspark.sql.functions import window
stringDF = spark.readStream.format("socket")
                        .option("host", "localhost")
                        .option("port", 9999)
                        .option("includeTimestamp", "true")
                        .load()
windowedCounts = stringDF.groupBy(
    window(stringDF.timestamp, "2 minutes", "1 minute"),
    stringDF.value).count()
query = windowedCounts.writeStream
                        .outputMode("update")
                        .format("console").start()
```

If a query is run in update output mode, the system keeps updating aggregations in a time window only.

To start a process type

```
aaa
```

in netcat window and press Enter key.

The first Batch 0: is empty.

Batch 1: contains

```
Batch: 1
-----
+-----+-----+-----+
|           window|value|count|
+-----+-----+-----+
|{2024-06-10 11:46...|  aaa|    1|
|{2024-06-10 11:47...|  aaa|    1|
+-----+-----+-----+
```

The results consist of two rows. An aggregation performed at the original position of a time window at 11:46 and an aggregation performed after a window slide one minute forward at 11:47.

Next, wait 1 minute and type

```
bbb
```

in netcat window and press Enter key.

Batch 2: contains

```
Batch: 2
-----
+-----+-----+-----+
```

window	value	count
{2024-06-10 11:47...}	bbb	1
{2024-06-10 11:48...}	bbb	1

The results consist of two rows. Only new data items are aggregated. An aggregation performed at the original position of a time window at 11:47 and n aggregation performed after a window slide one minute forward at 11:48.

Next, wait 1 minute and then quickly type

```
aaa
aaa
bbb
```

in netcat window and press Enter key to fit 3 data items in the same time window.

Only data item recorded in the same time window are aggregated.

Batch 3: contains

Batch: 3

window	value	count
{2024-06-10 11:48...}	aaa	1
{2024-06-10 11:49...}	aaa	1

Batch 4: contains

Batch: 4

window	value	count
{2024-06-10 11:48...}	aaa	2
{2024-06-10 11:49...}	bbb	1
{2024-06-10 11:49...}	aaa	2
{2024-06-10 11:48...}	bbb	2

The latest aggregation is performed 11:49 and it contains aggregation of data entered the most recently, i.e two aaa and one bbb.
